

En quête de parallélisme : Tomasulo, du hardware au thread

Motivation:

Le parallélisme est désormais la ressource vers laquelle se tourne le HPC (High Performance computing), notamment depuis l'introduction, suite au IA, de carte 'graphique' dédié au calcul, et la récente introduction prometteuse des attributs de fonction en C. Pourtant, son utilisation reste faible pour cause de difficulté de développement. C'est pourquoi je cherche à automatiser sa mise en place.

Ancrage:

Ce TIPE s'intéresse à l'introduction, semi automatisé, de technique de calcul dans un programme. Cela peut être vu comme une transformation du programme/code source, qui est précisément l'un des thèmes de l'année

Positionnement thématique:

- Technologies informatiques
- Informatique Théorique
- Informatique pratique

Biblio commentée

Depuis des années, les architectures de processeur sont de plus en plus complexes, jusqu'à des monstres de complexité prenant plusieurs centaines de pages pour être décrits [1]. L'un des facteurs non négligeable de cette complexité est l'introduction de l'exécution dite « hors ordre » (out of order en anglais)

Cette exécution hors ordre et notamment permise par l'utilisation de l'algorithme de Tomasulo [2], utilisant notamment un plus grand nombre de registre physiques qu'il n'y a de registre adressable, afin de supporter deux opérations indépendantes utilisant le même registre physique en leur donnant accès à des registres physiques différents.

Cependant, toutes ces optimisations internes au processeur ne sont pas (ou du moins pas directement) accessibles au programmeur, pour des raisons de compatibilité du jeu d'instructions, menant aujourd'hui à une double dette technique : du point de vue des processeurs, s'efforçant de rester compatible avec une architecture ancienne et des programmes écrits pour ces architectures, mais aussi du point de vue du langage, avec les langages les plus « bas niveau » possible, tel que le C, incapable de donner accès à toutes ces possibilités, si ce n'est en se basant sur des heuristiques potentiellement changeantes selon les processeurs individuels [3].

Cependant, comme le montre la quasi absence (ou du moins le très fort ralentissement) de progrès de performance monocœur dans les processeurs les plus récents, des limites physiques sur la vitesse et des limites de rentabilité sur la complexité/performance commencent à mettre un terme à la loi de Moore comme on la connaît. Cependant, elle prend une nouvelle dimension, avec la multiplication du nombre de cœurs disponibles.

Toutefois, pour profiter de cette possibilité, il faut ici faire l'effort de l'explicitier dans le programme, tâche complexe et longue, surtout si elle implique la modernisation d'une ancienne base de code sur laquelle il peut s'avérer nécessaire de modifier l'ensemble du code pour pouvoir profiter des performances offertes par le multi-threading. C'est pourquoi plusieurs projets tels que [4], [5] ou [6] tentent d'automatiser cette tâche mais présentent de nombreux défauts, les plus importants étant de se baser sur des sections explicitement marquées par le programmeur, et de n'être capable que d'optimiser du parallélisme relativement large (très faible voire aucune dépendance dans l'ensemble à paralléliser).

Le problème d'optimisation potentiellement automatique, et plus fine, va peut être remis au goût du jour avec l'introduction dans le récent standard C23 [7] d'attributs annotant les fonctions, simple et propagable (donc ne nécessitant pas forcément plus de marquage que dans les header des bibliothèques) permettant d'obtenir des informations sur les possibilités de modification de l'état global/des pointeurs par la fonction, et donc une possible mise en place de parallélisme plus fin avec indication à la compilation des dépendances aujourd'hui analysées, de manière très coûteuse, soit par le programmeur, soit par le matériel durant l'exécution (dans l'algorithme de Tomasulo)

[1]: Agner Fog | The microarchitecture of Intel, AMD, and VIA CPUs ;An optimization guide for assembly programmers and compiler makers | <https://www.agner.org/optimize/microarchitecture.pdf>

[2] : John Savard | Pipelined and Out-of-Order Execution (dont une partie sur l'algorithme de tomasulo) | <http://www.quadibloc.com/comp/cp07.htm>

[3] Association for Computing Machinery | C Is Not a Low-level Language ; Your computer is not a fast PDP-11. | <https://queue.acm.org/detail.cfm?id=3212479>

[4] MIT et autre chercheur du projet Tapir | The Tapir/LLVM compiler | <http://cilk.mit.edu/tapir/>

[5] Taskflow (groupe github) | Taskflow | <https://github.com/taskflow/taskflow>

[6] OpenMP | OpenMP | <https://www.openmp.org>

[7] ISO/IEC JTC1/SC22/WG14 | Draft du standard C, première version post-C23 | <https://www.open-std.org/JTC1/SC22/WG14/www/docs/n3220.pdf>

Problématique :

En tenant compte des limitations (théorique et technique) de la transformation de programme, comment proposer une solution d'abstraction de multithreading utilisant les opportunités de parallélisme fin, en conservant une bonne performance par thread ?

Est-il possible, en théorie et en pratique, de gagner des performances grâce à une implémentation logicielle (multi-thread) de l'algorithme de Tomasulo ?

Objectifs :

Étudier le problème de l'impossibilité de preuve dans le cas général de l'équivalence de deux algorithmes Étudier les limites techniques (coût de recherche exponentiel, effets de bord mal maîtrisés, tests exhaustifs ...) Comprendre les objectifs de l'algorithme de Tomasulo et son implémentation (aux détails privés près) dans les processeurs actuels Si j'en ai le temps, je voudrais de plus trouver une spécification raisonnable (utilisable et réalisable) d'interface de bibliothèque utilisant Tomasulo au niveau logiciel, puis l'implémenter une telle bibliothèque et commenter les résultats de son utilisation